Task	Example Prompt	Output	Metric
1: IOI (Appendix F.2)	"When John and Mary went to the store, Mary gave a bottle of milk to"	"_John"	Logit difference

2: Docstring (Appendix H.1)	`def f(self, files, obj, state, size, shape, option):`
	`"""document string example`
	
	`:param state: performance analysis`
	`:param size: pattern design`
	`:param`
		"_shape"	Logit difference

3: Greater-Than (Appendix G)	"The war lasted from 1517 to 15"	"18" or "19" or ...or "99"	Probability difference
4: tracr-xproportion (Appendix I.1)	`["a", "x", "b", "x"]`	`[0, 0.5, 0.33, 0.5]`	Mean Squared Error
5: tracr-reverse (Appendix I.2)	`[0, 3, 2, 1]`	`[1, 2, 3, 0]`	Mean Squared Error
6: Induction (Section 4.2)	"Vernon *Durs*ley and Petunia *Durs*"	"ley"	Negative log-probability

Table 1: Five behaviors for which we have an end-to-end circuit from previous mechanistic interpretability work, plus Induction. We automatically rediscover the circuits for behaviors 1-5 in Section 4. Tokens beginning with space have a "_" prepended for clarity.

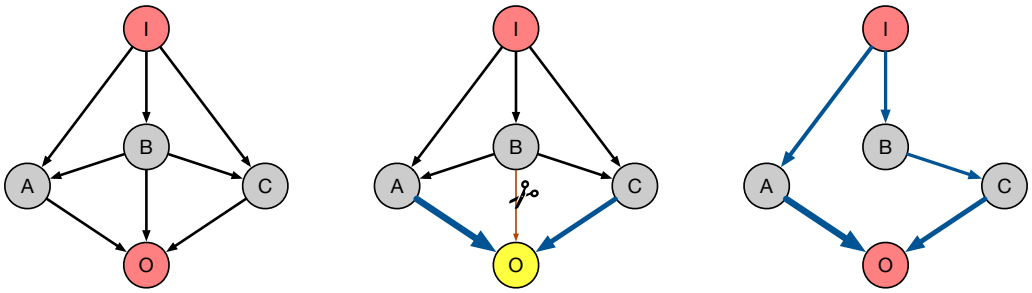
2.3 Step 3: Patch model activations to isolate the relevant subgraph

With the computational DAG specified, one can search for the edges that form the circuit. We test edges for their importance by using recursive *activation patching*: i) overwrite the activation value of a node or edge with a corrupted activation, ii) run a forward pass through the model, and iii) compare the output values of the new model with the original model, using the chosen metric (Section 2.1). One typically starts at the output node, determines the important incoming edges, and then investigates all the parent nodes through these edges in the same way. It is this procedure that ACDC follows and automates in Algorithm 1.

Patching with zeros and patching with different activations Activation patching methodology varies between mechanistic interpretability projects. Some projects overwrite activation values with zeros (Olsson et al., 2022; Cammarata et al., 2021), while others erase activations' informational content using the mean activation on the dataset (Wang et al., 2023). Geiger et al. (2021) prescribe *interchange interventions* instead: to overwrite a node's activation value on one data point with its value on another data point. Chan et al. (2022) justify this by arguing that both zero and mean activations take the model too far away from actually possible activation distributions. Interchange interventions have been used in more interpretability projects (Hanna, Liu, and Variengien, 2023; Heimersheim and Janiak, 2023; Wang et al., 2023), so we prefer it. However we also compare all our experiments to replacing activations with zeros (Section 4.2, Appendix E.2).

2.4 Explaining the circuit components

After successfully isolating a subgraph, one has found a circuit (Section 1). The researcher then can formulate and test hypotheses about the functions implemented by each node in the subgraph. There is early evidence that ACDC is helpful for making novel observations about how language models complete tasks, such as the importance of surprising token positions that help GPT-2 Small predict correctly gendered pronouns (Appendix K). In our work we focus on automating the time-consuming step 3 that precedes functional interpretation of internal model components, though we think that automating the functional interpretation of model components is an exciting further research direction.



(a) Choose computational graph, task, and threshold τ.

(b) At each head, prune unimportant connections.

(c) Recurse until the full circuit is recovered.

Figure 2: **How ACDC works** (Steps 2a-2c). Step 2a: a practitioner specifies a computational graph of the model, the task they want to investigate, and a threshold under which to remove connections. Step 2b: ACDC iterates over nodes in the computational graph, replacing activations of connections between a node and its children, and measuring the effect on the output metric. Connections are removed if their measured effect on the metric under corruption is below the threshold τ. Step 2c: recursively apply Step 2b to the remaining nodes. The ACDC procedure returns a subgraph of the original computational graph.

3 Automating circuit discovery (Step 3)

This section describes algorithms to automate Step 3 of the mechanistic interpretability workflow (Section 2.3). In all three cases, we assume that the 'task' being studied is defined by a set of prompts $(x_i)_{i=1}^n$ on which the model's predictions have a noticeable pattern (see Table 1 for examples) and a set of prompts $(x_i')_{i=1}^n$ where this task is not present. We then use the activations of the models on a forward pass on the points x_i' as corrupted activations (Section 2.3).

Automatic Circuit DisCovery (ACDC). Informally, a run of ACDC iterates from outputs to inputs through the computational graph, starting at the output node, to build a subgraph. At every node it attempts to remove as many edges that enter this node as possible, without reducing the model's performance on a selected metric. Finally, once all nodes are iterated over, the algorithm (when successful) finds a graph that i) is far sparser than the original graph and ii) recovers good performance on the task.

To formalize the ACDC process, we let G be a computational graph of the model of interest, at a desired level of granularity (Section 2.2), with nodes topologically sorted then reversed (so the nodes are sorted from output to input). Let $H \subseteq G$ be the computational subgraph that is iteratively pruned, and $\tau > 0$ a threshold that determines the sparsity of the final state of H.

We now define how we evaluate a subgraph H. **We let $H(x_i, x_i')$ be the result of the model when x_i is the input to the network, but we overwrite all edges in G that are not present in H to their activation on** x_i' (the corrupted input).[4] This defines $H(x_i, x_i')$, the output probability distribution of the subgraph under such an experiment. Finally we evaluate H by computing the KL divergence $D_{\text{KL}}(G(x_i)||H(x_i, x_i'))$ between the model and the subgraph's predictions. We let $D_{\text{KL}}(G||H)$ denote the average KL divergence over a set of datapoints. Appendix C discusses alternatives to the KL divergence, and Appendix E.1 explores the consequences of optimizing the task-specific metrics from Table 1 instead.

Algorithm 1 describes ACDC. The order in which we iterate over the parents w of v is a hyperparameter. In our experiments the order is lexicographically from later-layer MLPs and heads to earlier-layer MLPs and heads, and from higher- to lower-indexed heads. We note that in one case in our work, the order of the parents affected experimental results (Appendix J).

Subnetwork Probing (SP; Cao, Sanh, and Rush, 2021). SP learns a mask over the internal model components (such as attention heads and MLPs), using an objective that combines accuracy and

[4]To implement the computation of $H(x_i, x_i')$, we initially run a forward pass with the unmodified model on the input x_i' and cache all activations.

Algorithm 1: The ACDC algorithm.

Data: Computational graph G, dataset $(x_i)_{i=1}^n$, corrupted datapoints $(x_i')_{i=1}^n$ and threshold $\tau > 0$.

Result: Subgraph $H \subseteq G$.

```
H ← G                                  // Initialize H to the full computational graph
H ← H.reverse_topological_sort()       // Sort H so output first
for v ∈ H do
    for w parent of v do
        H_new ← H \ {w → v}            // Temporarily remove candidate edge
        if D_KL(G||H_new) − D_KL(G||H) < τ then
            H ← H_new                  // Edge is unimportant, remove permanently
        end
    end
end
return H
```

sparsity (Louizos, Welling, and Kingma, 2018), with a regularization parameter λ. At the end of training, we round the mask to 0 or 1 for each entry, so the masked computation corresponds exactly to a subnetwork of a transformer. SP aims to retain enough information that a linear probe can still extract linguistic information from the model's hidden states. In order to use it to automate circuit discovery, we make three modifications. We i) remove the linear probe, ii) change the training metric to KL divergence as in Section 2, and iii) use the mask to interpolate between corrupted activations and clean activations (Section 3) rather than zero activations and clean activations. Appendix D.1 explains the details of these changes.

Head Importance Score for Pruning (HISP; Michel, Levy, and Neubig, 2019). HISP ranks the heads by importance scores (Appendix D.2) and prunes all the heads except those with the top k scores. Keeping only the top k heads corresponds to a subnetwork that we can compare to ACDC. We plot the ROC obtained from the full possible range of k. Like SP, this method only considers replacing head activations with zero activations, and therefore we once more generalize it to replace heads and other model components with corrupted activations (for details, see Appendix D.2).

4 Evaluating Subgraph Recovery Algorithms

To compare methods for identifying circuits, we seek empirical answers to the following questions.

- **Q1:** Does the method identify the subgraph corresponding to the underlying algorithm implemented by the neural network?
- **Q2:** Does the method avoid including components which do not participate in the elicited behavior?

We attempt to measure **Q1** and **Q2** using two kinds of imperfect metrics: some grounded in previous work (Section 4.1), and some that correspond to stand-alone properties of the model and discovered subgraph (Section 4.2).

4.1 Grounded in previous work: area under ROC curves

The receiver operating characteristic (ROC) curve is useful because a high true-positive rate (TPR) and a low false-positive rate (FPR) conceptually correspond to affirming **Q1** and **Q2**, respectively.

We consider *canonical* circuits taken from previous works which found an end-to-end circuit explaining behavior for tasks in Table 1. We formulate circuit discovery as a binary classification problem, where edges are classified as positive (in the circuit) or negative (not in the circuit). Appendices F, G, H, I and J describe and depict the canonical circuits for each task. Appendix E.3 considers the node classification problem instead, which is less appropriate for ACDC but more appropriate for other methods.